UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

CARRERA:

ING. TECNOLOGÍAS DE LA INFORMACION

NRC: 29081

ASIGNATURA:

PROGRAMACION INTEGRATIVA DE COMPONENTES

PROYECTO P1

NOMBRES:

CRISTOPHER LASLUISA

INSUASTI SAUL

AYALA VANESSA

YUGSI JORGE

VASCONEZ CHRISTIAN

DOCENTE: ING. VILMER CRIOLLO

FECHA: 12 DE NOVIEMBRE DEL 2025

Introducción

El desarrollo web moderno se orienta hacia la modularidad, reutilización y encapsulamiento del código. En este contexto, los Web Components representan una de las tecnologías más importantes para crear interfaces web dinámicas y personalizadas.

El presente proyecto tiene como finalidad aplicar los principios fundamentales de los Web Components mediante la creación de una mini biblioteca que agrupe componentes reutilizables, semánticos y funcionales para una interfaz de tareas denominada PRY_TASKS.

Objetivo General

Aplicar los conceptos y especificaciones de los Web Components (Custom Elements, Shadow DOM, ES Modules y HTML Templates) para desarrollar una colección modular y funcional de componentes web reutilizables

Objetivos Específicos

- Implementar componentes personalizados mediante Custom Elements, asegurando encapsulamiento con Shadow DOM y estructura mediante HTML Templates.
- Integrar varios componentes que interactúen entre sí dentro de una interfaz funcional, utilizando ES Modules para su modularización y carga dinámica.

Desarrollo

El proyecto PRY_TASKS tiene como objetivo principal demostrar la capacidad de crear una interfaz dinámica y funcional usando los principios de los Web Components. Está estructurado para mostrar la gestión de tareas, incluyendo la creación, visualización y seguimiento de progreso de cada una.

1. Estructura del Proyecto

La estructura general del proyecto sigue una organización modular dentro de la carpeta PRY_TASKS:

```
/proyecto-web-components
│
├── index.html
├── public/
│   ├── js/
│   │   ├── main.js
│   │   ├── TaskItem.js
│   │   ├── TaskList.js
│   │   ├── ProgressBar.js
│   └── vendor/bootstrap/
│       └── css/
```

- **index.html:** archivo principal donde se importan los módulos y se estructura la interfaz.
- **main.js:** punto de entrada que importa los componentes y configura su interacción.
- **TaskItem.js, TaskList.js, ProgressBar.js**: módulos independientes que representan cada componente.

2. Componentes Personalizados

a. TaskItem.js

Este archivo define el componente <task-item>, el cual representa una tarea individual dentro del sistema.

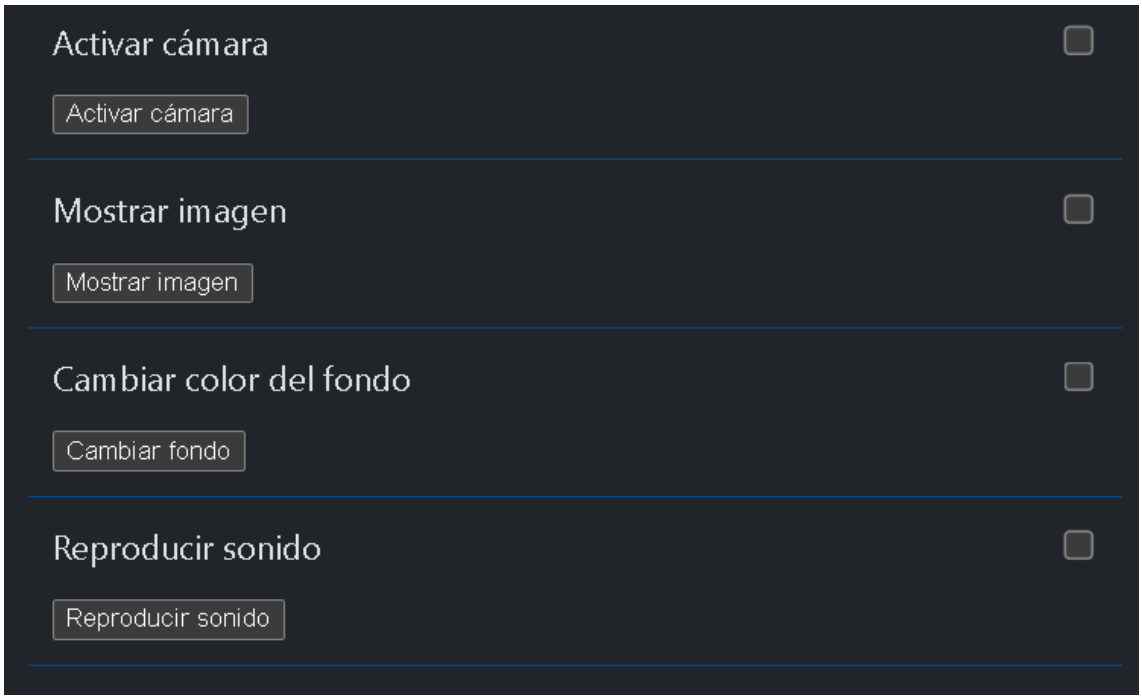
Utiliza el Shadow DOM para encapsular sus estilos y estructura.

Emplea un template HTML que contiene el nombre de la tarea y botones de control (completar o eliminar).

Implementa eventos personalizados para notificar al componente padre sobre cambios en el estado de la tarea.

Ejemplo de funcionamiento:

Cada vez que el usuario marca una tarea como completada, se dispara un evento que actualiza el progreso global mostrado en la barra de progreso.



b. TaskList.js

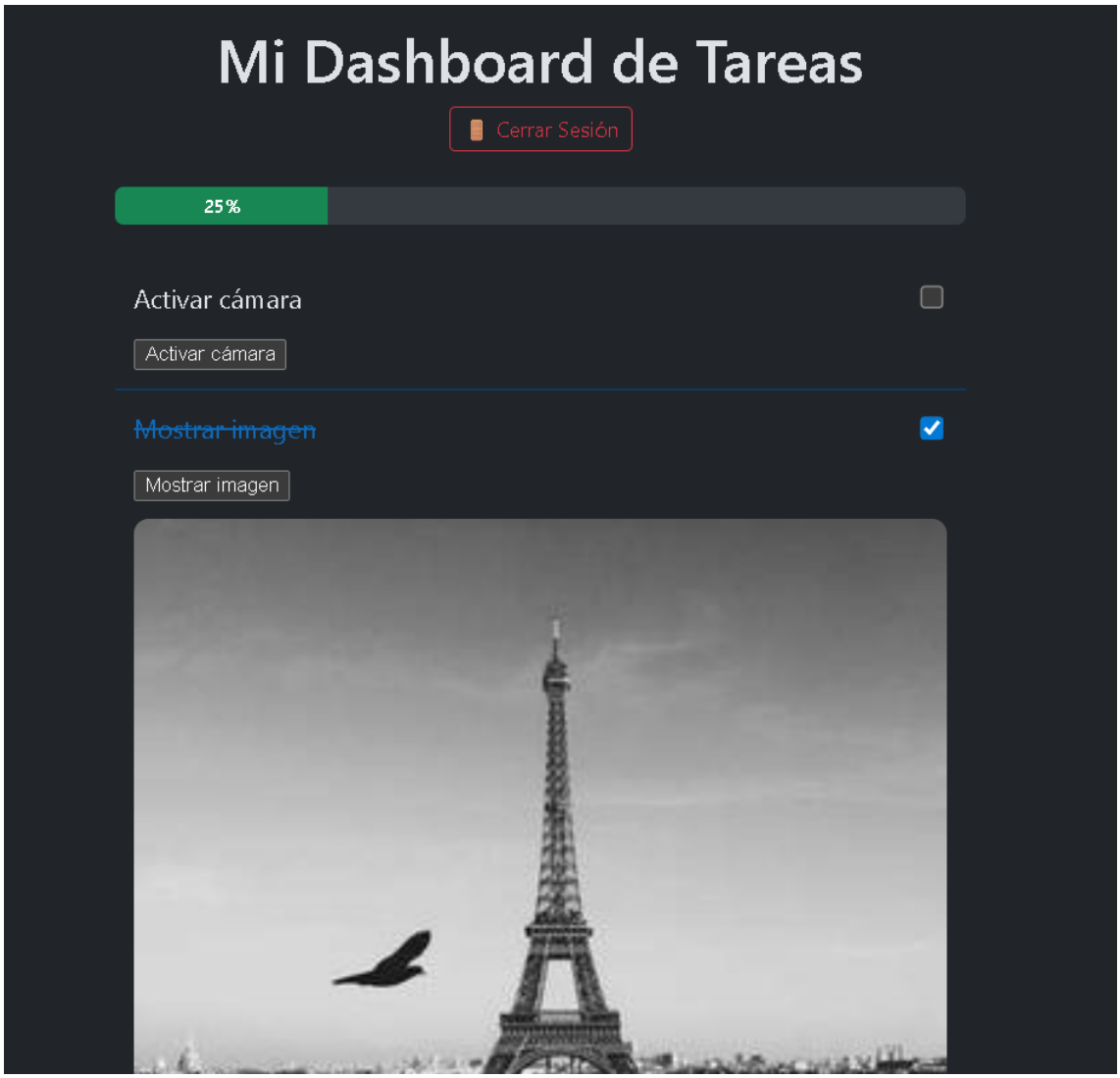
Define el componente <task-list>, que actúa como contenedor de múltiples tareas.

Gestiona la creación de nuevos <task-item>.

Escucha los eventos generados por cada tarea individual (como completar o eliminar).

Actualiza la lista de tareas en tiempo real y mantiene sincronía con el componente de progreso.

Además, este módulo se encarga de renderizar dinámicamente las tareas agregadas y comunicarse con el ProgressBar mediante eventos globales o propiedades compartidas.



c. ProgressBar.js

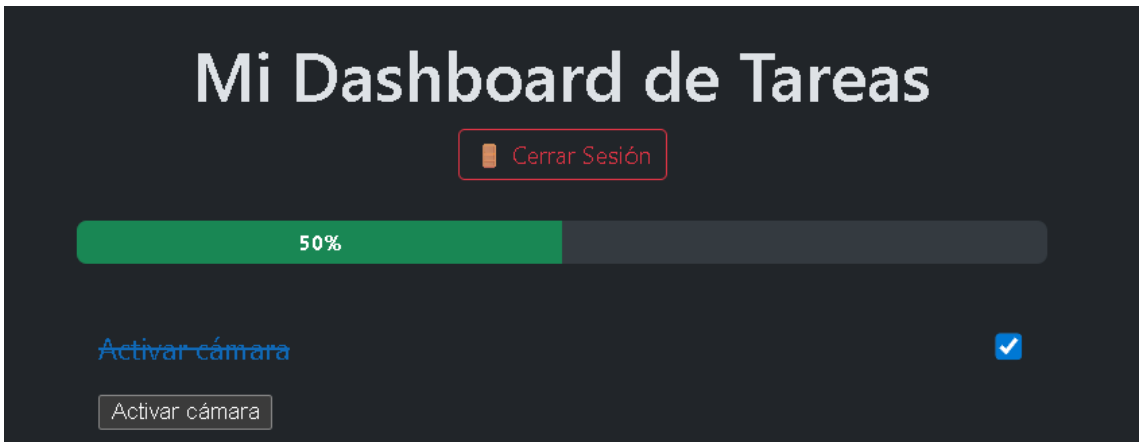
Contiene el componente <progress-bar>, que visualiza gráficamente el avance de las tareas completadas.

Usa Shadow DOM para encapsular sus estilos y evitar conflictos de CSS.

Recibe información desde <task-list> para calcular el porcentaje de progreso.

Actualiza la barra en tiempo real conforme cambian los estados de las tareas.

El comportamiento reactivo del componente permite que la interfaz se actualice de forma automática sin recargar la página.



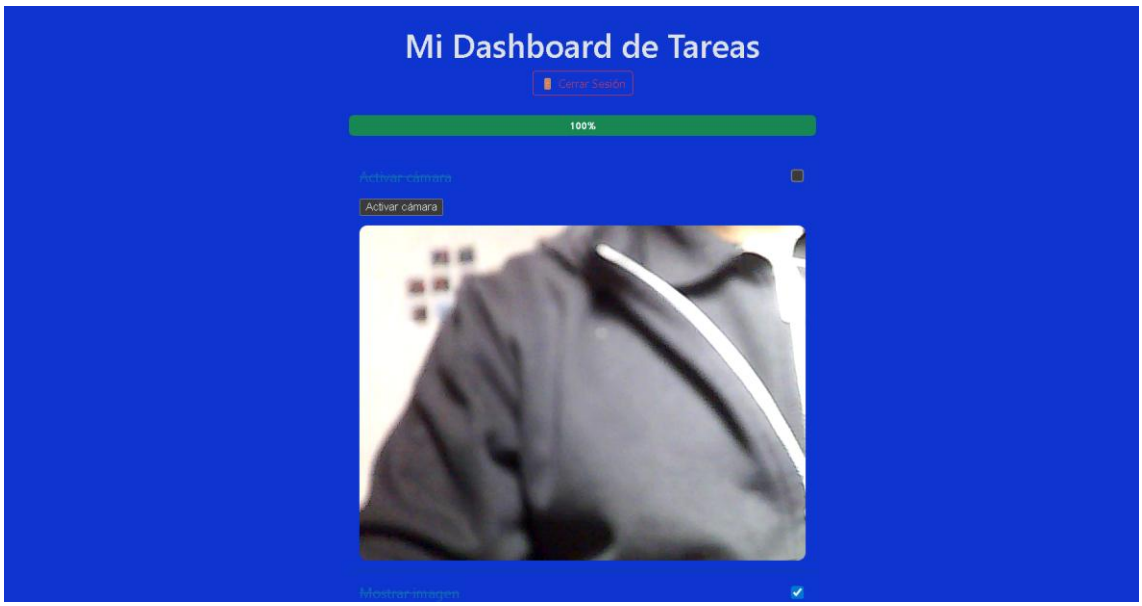
d. main.js

El archivo main.js cumple la función de orquestador del sistema.

Importa los tres componentes mediante ES Modules.

Inicializa el entorno y garantiza que los elementos personalizados estén definidos antes de ser usados.

Permite la comunicación indirecta entre los componentes, asegurando la cohesión del sistema.



3. Flujo de Interacción entre Componentes

El flujo de funcionamiento se desarrolla de la siguiente manera:

El usuario agrega una tarea en el formulario principal.

El componente <task-list> la inserta como un nuevo <task-item>.

Cuando el usuario marca una tarea como completada, <task-item> emite un evento al <task-list>.

<task-list> recalcula el progreso y actualiza el componente <progress-bar>.

Todo el proceso ocurre sin refrescar la página, gracias al enlace de datos mediante eventos.

4. Aspectos Técnicos Destacables

Uso correcto de Custom Elements mediante customElements.define().

Encapsulamiento total con Shadow DOM, evitando colisiones de estilo.

Modularización con ES Modules, separando la lógica en archivos independientes.

Definición estructurada mediante HTML Templates, asegurando claridad en la interfaz.

Integración visual con Bootstrap, garantizando un diseño responsivo y limpio.

```html
            <div class="progress mb-4" style="height: 25px;">
                <div id="progress-bar"
                     class="progress-bar bg-success fw-bold"
                     style="width: 0%;">0%</div>
            </div>


            <task-list class="mb-4">

                <task-item task="tarea1">Activar cámara</task-item>
                <task-item task="tarea2">Mostrar imagen</task-item>
                <task-item task="tarea3">Cambiar color del fondo</task-item>
                <task-item task="tarea4">Reproducir sonido</task-item>




            </task-list>



            <video id="video" autoplay style="display:none; width:100%;"
                   class="rounded mb-3"></video>

            <img id="imagenDemo" src="https://picsum.photos/300"
                 style="display:none; width:100%;" class="rounded mb-3">

        </div>
    </main>
```

Conclusiones

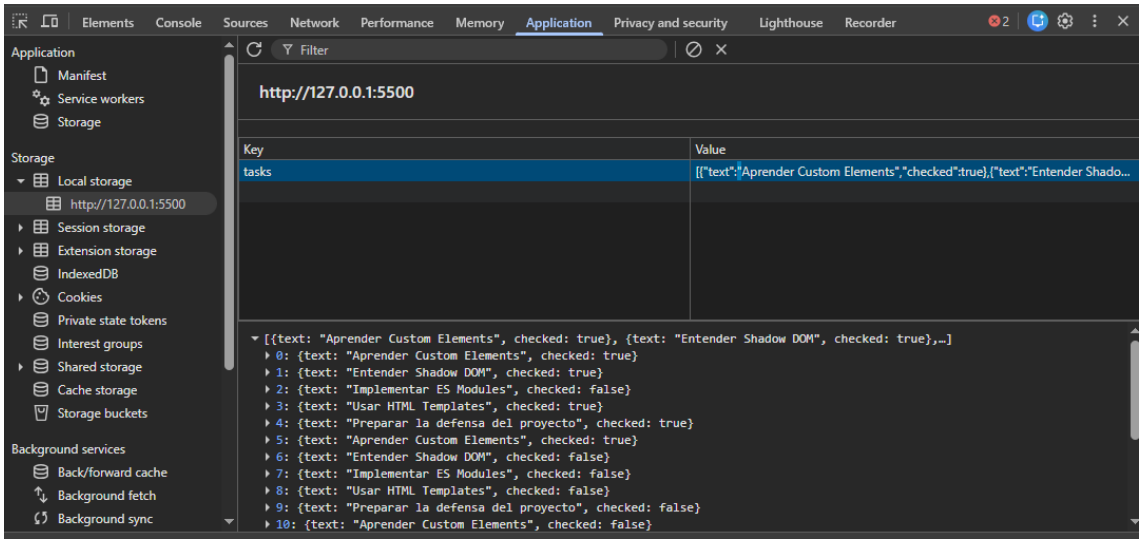
- El Proyecto PIC cumple satisfactoriamente con los objetivos propuestos, demostrando el dominio de las tecnologías que componen los Web Components.
- El uso de Custom Elements, Shadow DOM, HTML Templates y ES Modules permite construir componentes altamente reutilizables, con un comportamiento independiente y comunicación eficiente.
- La arquitectura obtenida promueve la mantenibilidad del código y la escalabilidad del sistema, sentando las bases para proyectos web modernos y sostenibles.

5. Persistencia de Datos con LocalStorage

Para complementar la funcionalidad del sistema PRY_TASKS, se implementó el almacenamiento persistente de las tareas mediante la API LocalStorage del navegador. De esta manera, las tareas puestas o marcadas como completadas se mantienen incluso después de recargar o cerrar la página. La lógica de persistencia se desarrolló dentro del componente <task-list>, responsable de coordinar todos los <task-item>. Cada vez que el usuario marca o desmarca una tarea, el componente ejecuta un método que guarda en LocalStorage un arreglo con el texto y el estado de cada tarea. Al iniciar la aplicación, el componente carga automáticamente las tareas

almacenadas. Gracias a esta implementación, la lista de tareas se conserva entre sesiones.

Para verificar se aplasta F12 luego pestaña Application y se coloca en Local Storage y nos aparecerá la clave llamada “tasks” como se puede observar a continuación:



Recomendaciones

- Incluir almacenamiento persistente (LocalStorage o IndexedDB) para mantener las tareas tras recargar la página.
- Incorporar más componentes interactivos, como <task-filter> o <task-summary>.
- Mejorar la accesibilidad (A11y) mediante atributos ARIA y etiquetas semánticas.
- Documentar la API de cada componente para facilitar su reutilización en otros entornos.
- Añadir pruebas automáticas (con Jest o Web Test Runner) para validar el correcto funcionamiento.

Referencias Bibliográficas

Mozilla Developer Network (MDN). (2024). Using Web Components. Recuperado de: https://developer.mozilla.org/en-US/docs/Web/Web_Components

W3C. (2023). Custom Elements Specification. Recuperado de: https://www.w3.org/TR/custom-elements/

Google Developers. (2024). Introduction to Shadow DOM. Recuperado de: https://developers.google.com/web/fundamentals/web-components/shadowdom

WHATWG. (2023). HTML Living Standard — Templates and Slots. Recuperado de: https://html.spec.whatwg.org/